

README

DZ-05 — Authenticate sensitive HelixLLM API endpoints

Run-id: dz05_endpoint_auth_20260707 **Date:** 2026-07-07 **Branch:** feature/helixllm-full-extension **Anchor:** §11.4.10 (secret/auth), §11.4.108 (runtime signature), §11.4.115 (RED → GREEN), §11.4.5 (captured evidence), §11.4.74 (reuse existing middleware)

Finding

The gateway API-key middleware (`gwmw.APIKeyAuth`) was applied **only** to the gateway's own `/v1` RouterGroup (`internal/gateway/router.go:63-64`). Four other route groups register on **separate** `gin.RouterGroup` instances that do NOT inherit that middleware, so they were reachable **unauthenticated** even with `HELIX_AUTH_API_KEYS` configured: - `/v1/agents/*` + `/v1/cache/stats` — agent control (LLM chat, tool execution, coordinate/plan, memory R/W) - `/internal/cluster/*` — control plane (status/probe + **SSH deploy/rebalance** across hosts) - `/internal/knowledge/*` — vector-store data plane (ingest/query/collections/stats)

Per-route-group classification

Group	Classification	Action
<code>/internal/cluster/*</code>	real-exposure (SSH deploy/rebalance)	FIXED — gated with <code>gwmw.APIKeyAuth</code>
<code>/v1/agents/*</code> + <code>/v1/cache/stats</code>	real-exposure (agent + tool control)	FIXED — gated with <code>gwmw.APIKeyAuth</code>
<code>/internal/knowledge/*</code>	real-exposure (data ingest/query)	FIXED — gated with <code>gwmw.APIKeyAuth</code>
<code>/ws</code>	design-question	NOT fixed — header-based auth breaks browser WS clients

Group	Classification	Action
		(§11.4.6); operator decision needed on WS credential channel
/metrics, /internal/metrics, /internal/health	intentionally-public	documented, left public (scraper + liveness convention)

Fix

Reused the EXISTING `internal/gateway/middleware.APIKeyAuth` (no new auth scheme, §11.4.74). The three registrars gained an optional `authMW ...gin.HandlerFunc` applied via `group.Use(authMW...)`; `cmd/helixllm/main.go` passes `gwmw.APIKeyAuth(cfg.Auth.APIKeys)` (same value as `/v1`). Empty keys $\Rightarrow$ open-access (identical to `/v1`); keys configured $\Rightarrow$ enforced. Variadic keeps all existing callers/tests (incl. `tests/integration`) compiling unchanged.

Runtime signature (§11.4.108) — see `red_green_evidence.txt`

- **RED_MODE=1** (open wiring, reproduces pre-fix exposure): `unauth GET /internal/cluster/status` $\rightarrow$ **200**, `/internal/knowledge/stats` $\rightarrow$ **200**, `/v1/agents/tools` $\rightarrow$ **200** (defect present).
- **RED_MODE=0** (fixed wiring): `unauth` $\rightarrow$ **401**, valid `Authorization: Bearer <key>` $\rightarrow$ **200** (defect absent).
- `go build ./... EXIT 0`; full suites of `control/knowledge/agents/gateway/gateway-middleware/integration` all ok.

Standing regression guards (§11.4.115 polarity switch)

- `internal/control/auth_dz05_test.go` — `TestDZ05_ClusterControlRequiresAuth`
- `internal/knowledge/auth_dz05_test.go` — `TestDZ05_KnowledgeRequiresAuth`
- `internal/agents/auth_dz05_test.go` — `TestDZ05_AgentsRequireAuth`